

プログラミングの基礎 2

学習目標

- 制御構文を使えるようになる。
- 自作関数を宣言して使えるようになる
- クラスを宣言して使えるようになる。

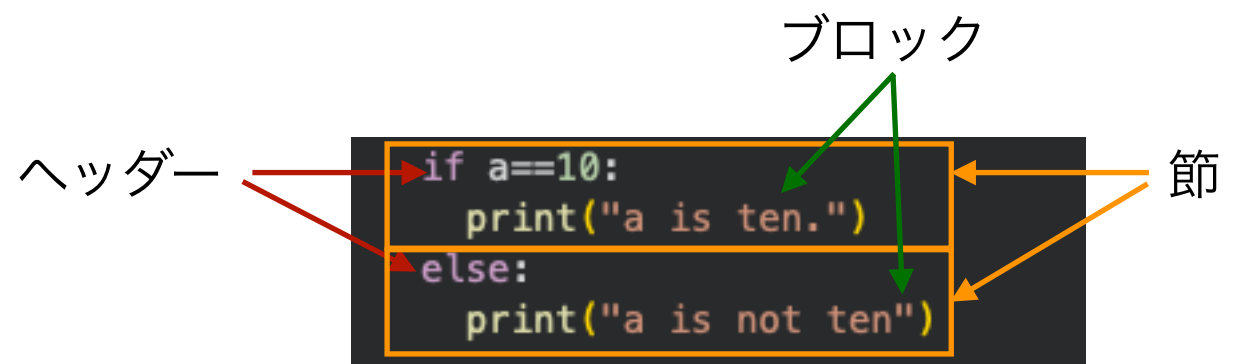
制御構文とは

● 制御構文

- 条件による分岐や繰り返し構文などプログラムの流れを制御する。
- 条件分岐：条件によってプログラムの処理を変更する。
- 繰り返し構文：特定の条件で処理を繰り返す。

● 複合文

- 複合文：中に複数の文を持っており、それらを実行するかを制御する文
 - 制御構文、関数定義、クラス定義など。。。
 - 1つ以上の節を持つ。
 - 節はヘッダーとブロックをもつ。



条件分岐

◎ 条件分岐

- 条件によって処理を分岐する (if...else...)
- ヘッダーに条件分岐のためのルールを書く。
- それぞれの場合の処理をブロックに書く。
- ブロックはインデントで字下げする。
- 多岐分岐
 - if, elif, elseを使う。
 - 最初に合致した条件のブロック1つが実行される。
- 入れ子構造
 - 同じ階層の複合文のブロックはインデントを揃える。
 - インデントを間違える全く違う処理となることに注意！！

```
a=10
if a==10:
    print("a is ten.")
else:
    print("a is not ten")

a is ten.
```

```
a=10
if a<10:
    print("a is smaller than ten")
elif a>10:
    print("a is larger than ten")
else:
    print("a is ten")

a is ten
```

```
a=1
b=4
if a<2:
    print("a is smaller than two")
    if b==3:
        print("b is three")
    else:
        print("b is not three")
else:
    print("a is not smaller than two")

a is smaller than two
b is not three
```

繰り返し処理

◎ 繰り返し処理

- 条件に基づいて同じ実行を繰り返す (while, for...)
- 条件式が真の間繰り返す：while文
 - ヘッダーに書かれた条件が満たされている間、ブロックの内容を繰り返す
 - 条件式から外れる仕組みを入れないと無限ループになるので注意が必要
- リストの要素を順に実行する：for文
 - リストの要素を順番に仮変数に渡して繰り返す
 - リスト型以外にも文字列型、辞書型、タプル型でも同様に使える。（イテラブル型）

iにllの要素を順番に
代入して繰り返す

```
ll=["a","b","c","d"]
for i in ll:
    print(i)
```

```
a
b
c
d
```

```
i=0
while i<4:
    print("i=",i)
    i +=1
```

```
i= 0
i= 1
i= 2
i= 3
```

iをカウンタとして増やす

辞書型に対するfor文

```
dict1={"name":"Taro","age":15,"city":"Tokyo"}
for k in dict1:
    print(k)
```

key値を順番に処理

```
name
age
city
```

```
for v in dict1.values():
    print(v)
```

valueを順番に処理

```
Taro
15
Tokyo
```

```
for k,v in dict1.items():
    print(k+":"+str(v))
```

keyとvalueの組
みを順番に処理

```
name:Taro
age:15
city:Tokyo
```

繰り返し処理

- 決まった回数の実行をする：for文

- range関数を使う

0~4までのループ

```
for i in range(5):  
    print(i)
```

0
1
2
3
4

2~4までのループ

```
for i in range(2,5):  
    print(i)
```

2
3
4

0~4までのループ

ただし2ずつ増える

```
for i in range(0,5,2):  
    print(i)
```

0
2
4

- リスト内包表記

```
data=[4,7,1,100]  
data2=[i+1 for i in data]  
print(data2)
```

[5, 8, 2, 101]

[式 for 仮変数 in リスト]でリスト内の要素を順番に仮変数として式を評価した数値をリストにできる

繰り返し処理

◎ ループの制御

- break : for/whileループを中断して抜け出す

breakによりwhileループから抜け出せる

```
i=0
while(True):
    if i==5:
        break
    i+=1
    print(i)
```

1
2
3
4
5

- continue : 現在の周回の以後の処理をスキップして次の周回の処理に入る

iが偶数の場合はすぐに次の処理に入る

```
for i in range(1,10):
    if i % 2 ==0:
        continue
    print(i)
```

1
3
5
7
9

- ループが入れ子になっている場合にはbreak/continueは内側のループが対象となる

- ループの最後のelse文は全てのループが終了された後に実行される処理。（breakでループを抜けた場合には実行されない。）

```
data=[1,5,10,4,3]
x=2
for i in data:
    if i==x:
        print("Found")
        break
else:
    print("Not Found")
```

Not Found

```
x=10
for i in data:
    if i==x:
        print("Found")
        break
else:
    print("Not Found")
```

Found

例外処理

◎ 例外処理

- 例外：未然に防ぐことが難しいエラーのこと
- 例えばユーザの入力に対して何らかの処理を行うコードではユーザによる想定していない入力を受け取れるかもしれない

```
1 x=input("Please type integer:")  
2 print("2*x=",2*int(x))
```

Please type integer:

ユーザからの入力をxに代入する

```
x=input("Please type integer:")  
print("2*x=",2*int(x))
```

Please type integer:2
2*x= 4

入力が整数なら問題なく動く

```
x=input("Please type integer:")  
print("2*x=",2*int(x))
```

Please type integer:taro

入力が整数でない場合動かない

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipython-input-1742045485.py in <cell line: 0>()  
      1 x=input("Please type integer:")  
----> 2 print("2*x=",2*int(x))  
  
ValueError: invalid literal for int() with base 10: 'taro'
```


例外処理

- 例外処理：エラーが出ても実行を継続できるように処理する

- tryのブロックでエラーが出るかもしれない処理を書く

- “except 例外の種類 as 例外変数”としてブロック内に例外が出た時の処理を書く

- 例外が出るとexcept内の処理が実行されて、次の処理へ進む

- 代表的な例外種類:

- ValueError : 値が不正
- TypeError : 型が不正
- KeyError : 存在しないkeyにアクセス
- IndexError : リストへの範囲外アクセスなど

```
x=input("Please type integer:")
try:
    print("2*x",2*int(x))
except ValueError as ex:
    print("Invalid input",ex)
print("Finish")
```

```
Please type integer:2
2*x 4
Finish
```

```
Please type integer:taro
Invalid input invalid literal for int() with base 10: 'taro'
Finish
```

```
my_dict={"taro":10,"jiro":20,"saburo":30}
try:
    print(my_dict["sakura"])
except KeyError as ex:
    print("Invalid key",ex)
print("Finish")
```

```
Invalid key 'sakura'
Finish
```

関数

- 関数：与えられた入力（引数）に対して出力（戻り値）を返すもの
- Pythonでは組み込み関数として関数が用意されている。

abs:絶対値
を返す関数



```
x=abs(-1)
print(x)
```

```
1
```

len:リストの長
さを返す関数



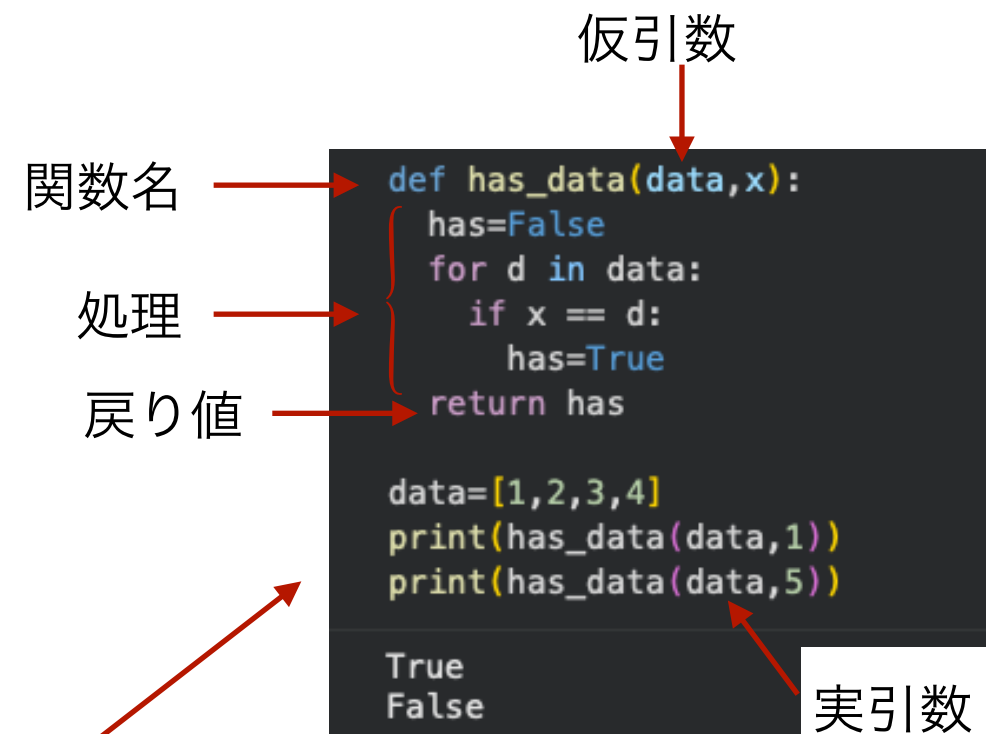
```
data=[1,2,4,7,100]
x=len(data)
print(x)
```

```
5
```

- さらにライブラリをimportすることでより多くの関数を使えるようになる。
- ここでは関数を自分で定義して使っていく方法を学ぶ。
- 関数を定義して使うメリットは？
 - 同じ（似たような）処理がコードの至るところで出現する場合、関数としてまとめておくことでコードが簡潔になる。また処理を変更したいときに関数の処理を変更するだけで良くなる。
 - 処理をある程度の括りでまとめることでコードの可読性が上がる。

関数

- 関数：与えられた入力（引数）に対して出力（戻り値）を返すもの
- 引数：関数の中で利用可能な変数
 - 仮引数：関数が受け取る変数（関数の中でのみアクセス可能）
 - 実引数：呼び出し元から関数へ渡される変数
- 関数はdefを使って宣言する
- returnで関数の処理を抜けて（あれば）戻り値を呼び出し元へ返す
- 戻り値なしでreturnすることもできる。



dataの中にxという要素があるかどうか調べる関数

関数

- スコープ：コードの中での変数の有効範囲
 - グローバル変数：コード全体からアクセス可能
 - ローカル変数：定義された関数の中でのみアクセス可能

関数の外で定義（グローバル変数）

関数の中で定義（ローカル変数）

グローバル変数は関数内で使える

ローカル変数は関数外では使えない

```
global_var=10
def scope_check():
    local_var=1
    print(global_var)
    print(local_var)

scope_check()
print(global_var)
print(local_var)
```

```
10
1
10
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-2349672714.py in <cell line: 0>()
      7 scope_check()
      8 print(global_var)
----> 9 print(local_var)

NameError: name 'local_var' is not defined
```

- 異なるスコープの変数は名前が同じでも違う変数として扱われる（ただし同じ名前の変数を使うとコードの可読性が低下するので使わない方が良い）

varはローカル変数として扱われグローバル
変数のvarとは区別される

```
var=1
def func():
    var=2
    print(str(var)+" in func")
func()
print(var)
```

```
2 in func
1
```

関数

- 関数の引数は参照の引渡し
 - 引数がミュータブルかイミュータブルを意識する。

```
def reset_list(data):  
    for i in range(0, len(data)):  
        data[i]=0  
  
data=[1,2,3]  
print(data)  
reset_list(data)  
print(data)
```

[1, 2, 3]
[0, 0, 0]

ミュータブル変数の引数は関数内での値の変更が関数の呼び出し元に反映される

```
def reset_int(x):  
    x=0  
  
x=1  
print(x)  
reset_int(x)  
print(x)
```

1
1

イミュータブル変数の引数は関数内での値の変更が関数の呼び出し元に反映されない

- 関数内でglobal宣言することで関数外の変数への代入ができる。

```
str_global="test1"  
def check_scope():  
    str_global="test2"  
  
check_scope()  
print(str_global)
```

test1

関数内での値の代入はあくまでローカル変数として扱われる

```
str_global="test1"  
def check_scope():  
    global str_global  
    str_global="test2"  
  
check_scope()  
print(str_global)
```

test2

global宣言をすることでglobal変数として扱われる

関数

- 関数の引数にデフォルト値を設定する

vaにデフォルト値0を設定

```
def reset_list(data, va=0):  
    for i in range(0, len(data)):  
        data[i]=va  
  
data=[1,2,2,3,4]  
reset_list(data,1)  
print(data)  
reset_list(data)  
print(data)  
  
[1, 1, 1, 1, 1]  
[0, 0, 0, 0, 0]
```

vaの入力を省略するとデ
フォルト値が使われる

デフォルト値が設定されて
いない引数は省略できない

- デフォルト値の評価は関数の定義時に行われる

```
par0="test1"  
def my_print(par=par0):  
    print(par)  
  
par0="test2"  
my_print()  
  
test1
```

デフォルト値として変数を受け取れる

デフォルト値として与えた変数を関数の定義後に書き
換えても、デフォルト値は関数定義時のものとなる

関数

- デフォルト値がmutableなオブジェクトの場合、変更が維持される。
→関数の呼び出し回数等に依存した結果になるのでこのような使い方はあまり良くない)

```
def my_append(v, list=[]):  
    list.append(v)  
    return list  
  
print(my_append(1))  
print(my_append(2))  
  
[1]  
[1, 2]
```

デフォルト値に空のリストを設定

my_appendでの変数追加の操作は以前の関数呼び出しの追加操作を維持する

- 関数の呼び出し時に引数の名前を明示的に指定できる（キーワード引数）

```
def print_name(family, given):  
    print(family, given)  
  
print_name(given="Taro", family="Yamada")  
  
Yamada Taro
```

キーワード引数では関数定義時の引数の順番を気にしなくてよい

関数

- 可変長引数：関数の定義時に引数の個数を決めないで、実行時に引数の個数を決められる引数

```
def connect_str(*strings, sep):  
    ret = ""  
    for string in strings:  
        ret = ret + sep + string  
    return ret  
  
print(connect_str("a", "b", "c", sep="/"))  
  
/a/b/c
```

可変長引数は引数の前に*をつけて宣言する
(可変長引数は1関数につき一つ)

可変長引数は関数内でtuple型として扱われる

可変長引数の後ろにはキーワード引数のみ使える

- **をつける可不特定のキーワードを引数にできる

```
def create_dict(**input):  
    ret=dict()  
    for key,value in input.items():  
        ret[key]=value  
    return ret  
  
d=create_dict(name="taro",age=10,sex="male")  
print(d)  
  
{'name': 'taro', 'age': 10, 'sex': 'male'}
```

不特定のキーワードを引数にする

関数呼び出しはキーワード引数を使う

関数内では辞書型として受け取る

関数

- 戻り値をカンマで区切って並べることで複数の戻り値を返せる。

```
def find_max(data):  
    max_value=data[0]  
    imax = 0  
    for i in range(0,len(data)):  
        if max_value<data[i]:  
            max_value=data[i]  
            imax=i  
    return max_value,imax  
  
data1=[-1,4,2,100,3]  
mv,im = find_max(data1)  
print("mv=",mv)  
print("im=",im)  
  
mv= 100  
im= 3
```

returnで複数の値を返す

関数の呼び出し元では変数を並べて受け取れる

```
r=find_max(data1)  
print(type(r))  
  
<class 'tuple'>
```

戻り値はタプル型

- 再帰関数：自分自身を呼び出す関数

```
def factorial(n):  
    if n!= 0:  
        return n*factorial(n-1)  
    return 1  
  
print(factorial(10))  
  
3628800
```

内部で自分自身を呼び出している

再帰関数呼び出しの終了点が必須

関数

- 高階関数
 - pythonでは関数自体をオブジェクトとして関数の引数・戻り値に使える

```
def rm_element(data,cond):  
    r = []  
    for x in data:  
        if cond(x):  
            r.append(x)  
    return r  
  
def cond1(x):  
    if x<0:  
        return True  
    return False  
  
def cond2(x):  
    if 0<x and x<2:  
        return True  
    return False  
  
data=[1,4.2,1.4,0.4,-1.4,0.1]  
print(rm_element(data,cond1))  
print(rm_element(data,cond2))
```

condは関数として与える

dataの要素のうちcondがTrueの要素のみのリストを返す

cond1,cond2を関数として宣言

cond1,cond2を引数として渡す

```
[-1.4]  
[1, 1.4, 0.4, 0.1]
```

クラス

- オブジェクト指向プログラミング
 - 変数やそれを操作する関数をまとめたオブジェクト（クラス）を中心にコードを組み立ていく手法
 - Pythonではオブジェクト指向の書き方が必須ではない。必要に応じてオブジェクト指向を意識したコードを書いていくのがよい。
- クラス：変数やそれを操作する関数（メソッド）をまとめた設計図
 - 初期化メソッド：クラスをインスタンス化したタイミングで呼ばれるメソッド（変数の初期化などに使う）
 - インスタンス変数：クラスに属する変数
 - メソッド：クラスに属する関数

Parameterというクラスをparameter
という名前でインスタンス化
“.”を使ってインスタンス
変数やメソッドを使う

```
class Parameter:
    name = ""
    value = ""
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def print(self):
        print(self.name + ":" + str(self.value))

if __name__ == "__main__":
    parameter = Parameter("time", 0)
    parameter.print()
    parameter.value = 1
    parameter.print()
```

time:0
time:1

classの定義

クラス

```
class Parameter:
    name = ""
    value = ""
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def print(self):
        print(self.name + ":" + str(self.value))
```

- Pythonではインスタンスごとに変数を追加できる。

parameter1にunitという変数を追加

parameter2にはunitという変数はないのでエラー

```
parameter1=Parameter("time",0)
parameter1.unit="sec"
print(parameter1.unit)
parameter2=Parameter("mass",0)
print(parameter2.unit)

sec
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_296/3656278451.py in <cell line: 0>()
      3 print(parameter1.unit)
      4 parameter2=Parameter("mass",0)
----> 5 print(parameter2.unit)

AttributeError: 'Parameter' object has no attribute 'unit'
```

- __slots__でインスタンス変数の追加を制限できる

__slots__でインスタンス変数を宣言する

宣言した変数はこれまで通り扱える

__slots__で宣言していない変数を追加しようとするとエラー

```
class Parameter:
    __slots__=["name","value"]
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def print(self):
        print(self.name + ":" + str(self.value))

if __name__=="__main__":
    parameter=Parameter("time",0)
    parameter.value=1
    print(parameter.value)
    parameter.unit="sec"

1
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipython-input-413/1869635088.py in <cell line: 0>()
     12 parameter.value=1
     13 print(parameter.value)
----> 14 parameter.unit="sec"

AttributeError: 'Parameter' object has no attribute 'unit'
```

クラス

- メソッドの宣言では第1引数にはself(インスタンス自身)を渡す。第2引数以降にメソッド独自の引数を書く。

```
class Parameter:
    __slots__=["name","value"]
    def __init__(self,name,value):
        self.name=name
        self.value=value

    def set_value(self,v):
        self.value = v

    def print(self):
        print(self.name + ":" + str(self.value))

if __name__=="__main__":
    parameter=Parameter("time",0)
    parameter.print()
    parameter.set_value(1)
    parameter.print()

time:0
time:1
```

メソッドの宣言

メンバ変数にはself.XXXでアクセス

メソッドは"クラス名.メソッド名"でアクセス

- メソッドを動的に追加することもできる

予め追加しておきたいメソッドを定義

クラスそのものに追加

インスタンスに追加

クラスに追加したメソッドは全ての
インスタンスから呼び出せる
インスタンスに追加したメソッドはその
インスタンスからのみ呼び出せる

```
import types

def print_value(self):
    print(self.value)

def print_name(self):
    print(self.name)

if __name__=="__main__":
    Parameter.print_value = print_value
    parameter1=Parameter("time",0)
    parameter1.print_name = types.MethodType(print_name,parameter1)
    parameter2=Parameter("mass",1)
    parameter1.print_value()
    parameter2.print_value()
    parameter1.print_name()
    parameter2.print_name()

0
1
time
-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_296/521770894.py in <cell line: 0>()
     15 parameter2.print_value()
     16 parameter1.print_name()
--> 17 parameter2.print_name()

AttributeError: 'Parameter' object has no attribute 'print_name'
```

クラス

- ここまでのクラスはインスタンス化したオブジェクトからメソッドを呼び出してきた。（インスタンスメソッドという）
- インスタンス化しなくても呼び出せるメソッド（クラスメソッド）がある。

クラスメソッドの先頭には@classmethodをつける

第1引数は自身のクラスを渡す（慣習的にclsとする）

クラスメソッドはインスタンス化しなくても呼ぶことができる

```
import math
class Area:
    @classmethod
    def circle(cls,r):
        return r*r*math.pi
    @classmethod
    def square(cls,a,b):
        return a*b
print(Area.circle(1))
print(Area.square(2,3))

3.141592653589793
6
```

- インスタンス化しなくても呼び出せる変数（クラス変数）

クラス変数の宣言

クラスメソッドはインスタンス化なしで呼べるため、内部で使える変数もクラス変数に限られる

```
import math
class Area:
    PI = math.pi
    @classmethod
    def circle(cls,r):
        return r*r*cls.PI
print(Area.circle(1))

3.141592653589793
```

クラス変数は読み取り専用変数（定数）またはクラス自体の状態を監視するものとして使うことが多い

演習問題

1. 整数 n と実数 x を引数として

$$\sum_{i=0}^n \frac{x^i}{i!}$$

を計算する関数を作成せよ。

2. 会社のシステムにおいて社員を管理するクラスとして以下の仕様を満たすEmployeeクラスを作成せよ。

◎インスタンス変数

- 社員の名前 (name)、社員番号 (id)、部署名 (department)、月収 (salary) ボーナス (bonus)

◎メソッド

- 初期化メソッドは引数として社員の名前と社員番号を受け取りインスタンス変数に保存。部署名、月収、ボーナスはクラスの初期化時は未定であることがわかるように初期化する。
- それぞれのインスタンス変数 (hoge) の値を得るメソッド (get_hoge)。部署名、月収、ボーナスが未定の時にアクセスした場合にはエラーメッセージを出して処理を終了する。
- 部署名、月収、ボーナスの値を設定するメソッド (set_hoge)。
- ボーナスを年に2回として年収 (月収 $\times 12$ + ボーナス $\times 2$) を得るメソッド (get_annual_salary)

3. 2で作成したクラスを要素とするlistとして (Employee_list) を作成せよ。Employee_listから年収が x 円以上の社員の名前と社員番号を出力する関数を作成せよ。